

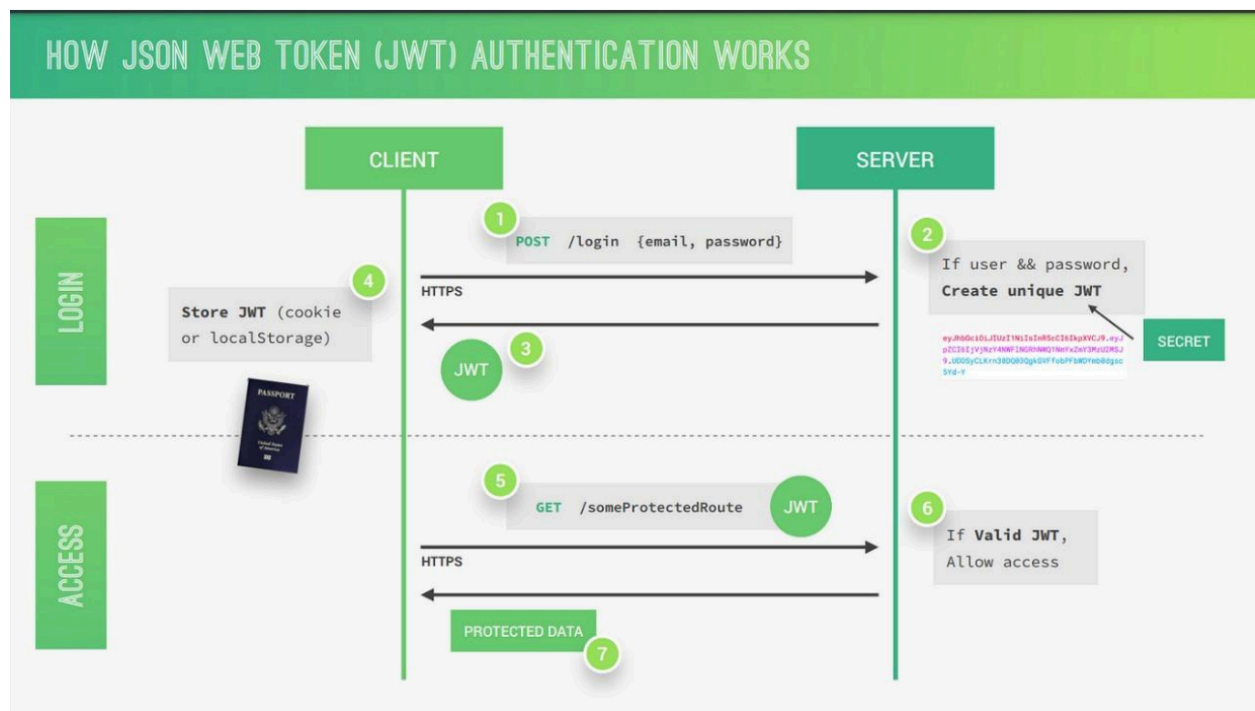
TASK 2 - Backend + JWT (Full Stack)

Overview

In this task, you will turn the local-only password vault from Task 1 into a real full-stack application. Everything that used to live in localStorage accounts, vault entries, and the "share link" will now live in a real backend with MongoDB, real password hashing, and real JWT-based authentication.

By the end, your Task 1 frontend will talk to a real server instead of pretending to.

Step 1: Understand JWT



Before touching any code, understand what a JWT actually is not just paste `jwt.sign()` because a tutorial said so.

A **JWT (JSON Web Token)** is a signed string the server hands a client after login, so the client can prove "I already logged in" on every future request without sending a password again.

A **JWT has three parts**, joined by dots: `header.payload.signature`

- **Header** - metadata: which algorithm signed the token (e.g. HS256) and that it's a JWT.
- **Payload** - the actual data, called "claims." For us that's things like the user's id and when the token expires (exp).

- **Signature** - created by the server using a secret key it never shares. This is what makes the token tamper-proof.

Important, and easy to miss: the header and payload are only encoded, not encrypted. Anyone holding the token can read them instantly paste any JWT into jwt.io and it decodes the payload for you. That means:

- Never put a password or anything sensitive inside the payload.
- The **signature** is the only thing stopping someone from editing the payload and handing back a fake token change even one character of the payload, and the signature no longer matches when the server checks it.

Why this matters for signup/login: on login, the server checks the password, then creates and signs a JWT containing the user's id, and sends it back. The client stores it and attaches it to every future request. The server never "remembers" who's logged in it just re-verifies the signature and expiry each time. That's what "stateless" auth means.

Terms worth locking down:

- Encode - reformat (base64url here). Reversible by anyone; not a security measure.
- Sign - attach proof, using a secret, that the content hasn't changed. This is the actual security.
- Claims - the data inside the payload (userId, exp, etc.).
- exp - the expiry claim. A valid signature on an expired token must still be rejected.

Concepts used: JWT structure (header.payload.signature), encoding vs signing, claims, token expiry

Resources to understand JWT better:

1. JWT structure (header, payload, signature) - [JSON Web Token](#),

 [JWT Explained In Under 10 Minutes \(JSON Web Tokens\)](#)

Step 2: Set up the backend and your data models

Now set up the real project. Initialize an Express server and connect it to MongoDB with Mongoose.

Create three models:

Model	Fields
User	_id, email, passwordHash, refreshTokenVersion, createdAt
VaultEntry	_id, userId, label, password, createdAt
ShareLink	_id, vaultEntryId, token, expiresAt, used, createdAt

At this stage, just get the server running and the models defined. No route logic yet.

Concepts used: Express project setup, MongoDB + Mongoose, schema design

Step 3: Implement signup and login with JWT

This is the core of the task implementing JWT in a real signup/login flow.

- Build POST /api/auth/signup: hash the incoming password with bcrypt before saving the user. Never store a plaintext password.
- Build POST /api/auth/login: check the submitted password against the stored hash using bcrypt.compare(), then issue a signed JWT (an access token) if it matches. You can use the real jsonwebtoken package here Step 1 already proved you understand what it does internally.

Test both routes with Postman before moving on. Don't wire up the frontend yet.

Concepts used: bcrypt hashing, JWT signing, Express route handlers

Step 4: Add refresh, logout, and route protection

Login should now issue two things: a short-lived access token (sent to the frontend) and a longer-lived refresh token (stored in an httpOnly cookie).

- Build POST /api/auth/refresh: reads the refresh token from the cookie and, if it's valid, issues a new access token.
- Build POST /api/auth/logout: invalidate the user's refresh token by bumping refreshTokenVersion.
- Write an authentication middleware that checks the access token on protected routes and attaches the logged-in user's identity to req.user.

Test the middleware on one dummy protected route before continuing.

Concepts used: access tokens vs refresh tokens, httpOnly/secure/sameSite cookies, Express middleware

Step 5: Build the real Vault API

Now build the actual vault routes, all protected by the middleware from Step 4:

- GET /api/vault — list only the logged-in user's entries.
- POST /api/vault — create a new entry for the logged-in user.
- DELETE /api/vault/:id — delete an entry, but only if it actually belongs to req.user.id.

This ownership check matters: "logged in" and "allowed to touch this specific entry" are two different questions, and your code needs to answer both.

Concepts used: protected routes, Mongoose queries scoped by userId, authorization vs authentication

Step 6: Build the real Share Link feature

- Build POST /api/vault/:id/share (protected): creates a ShareLink document with a cryptographically random token (crypto.randomBytes() not a sequential ID) and an expiry time, and returns the shareable URL.
- Build GET /api/share/:token (public no login required, and that's correct here, think about why): look up the ShareLink by token. If it hasn't been used and hasn't expired, show the password and set used: true. If it's already been viewed, say so instead of showing the password again.

Concepts used: crypto.randomBytes, resource lifecycle (unused → viewed/expired), designing a deliberately public endpoint

Step 7: Connect the React frontend to the real backend

Replace the Task 1 local-only logic with the real API.

- Add an AuthContext to hold the logged-in user and access token.
- Add a ProtectedRoute wrapper for /vault.
- Add an axios/fetch wrapper that catches an expired-access-token response, silently calls /api/auth/refresh, and retries the original request.
- Wire up your existing pages:
 - /login and /signup now call the real endpoints.
 - /vault now fetches from GET /api/vault instead of localStorage.
 - "Share" now calls POST /api/vault/:id/share and shows the real returned link.
 - /share-preview/:id is replaced by a real page hitting GET /api/share/:token.

Concepts used: React Context, protected routing, axios interceptors

Step 8 (optional bonus): Auto-expire share links

If you have time left, add a MongoDB TTL index on ShareLink.expiresAt so expired links get cleaned up automatically instead of just being checked-and-ignored in your route logic.

Concepts used: MongoDB TTL indexes

Useful Tips

- Keep your JWT secret in a .env file never hardcode it in a route file.
- Always use bcrypt.compare() to check a password. Never compare hashes with ===.
- Access tokens should be short-lived (minutes). Refresh tokens live longer and belong in an httpOnly cookie, not localStorage.

- Test every backend route with Postman/Thunder Client before touching the frontend if a route is broken, you want to know before React is also in the mix.
- Express middleware runs in the order you add it. A route protected by auth middleware needs that middleware listed before the route handler.
- If requests start failing with 401 partway through testing, check whether your access token simply expired that's exactly what Step 4's refresh flow exists to handle.
- Set up CORS with credentials: true on both the Express and axios/fetch sides if your frontend and backend run on different ports.
- req.user.id should always come from your verified token, never from anything the client sends in the request body that's what makes the ownership check in Step 5 mean something.

Deliverable

By the end of this task, you should have a fully working full-stack Password Vault: real signup and login backed by JWTs, an access/refresh token cycle, a Vault API that only ever shows a user their own entries, and a real one-time share link that works across devices all wired up to the frontend you built in Task 1.

Resources:

- **Base64url encoding** - [MDN Docs](#)
- **React, mongo, express, bcrypt, jwt complete tut - (vv good)**
 - ▶ Build a Fullstack Login & Register Page with MERN (react.js, mongo, express, bcrypt, jwt &...
- **HMAC-SHA256** - [Node.js crypto - createHmac](#),
 - ▶ HMAC explained | keyed hash message authentication code
- **Node.js crypto module** -
 - ▶ Crypto Module in Node.js | Unlocking Node.js #11 ,
 - ▶ Cryptography - Node.js Basics Part 8
- **bcrypt password hashing** - [Bcrypt npm Doc](#),
 - ▶ #34 Spring Security | Bcrypt Password Encoder
- **httpOnly / secure / sameSite cookies** - [HTTP Cookie MDN Doc](#),
 - ▶ Hacking HTTP Only, Same Site, Secure cookies with XSS? ,
 - ▶ Password Hashing & Verification with Bcrypt.js in Node.JS & Express JS
- **Express middleware** - [Express Middleware Doc](#),
 - ▶ Learn Express Middleware In 14 Minutes ,
 - ▶ Express Middleware
- **MongoDB + Mongoose schema design** - [MongoDB Schema and Models](#),
 - ▶ Mongoose JS Tutorial - 2 - Schema Design ,
 - ▶ Mongoose Schemas & Models Explained (The Blueprint for Your Data) | Part #98
- **Authentication vs authorization (ownership checks)** - [authorization cheatsheet](#),
 - ▶ Authentication vs Authorization Explained ,
 - ▶ Authentication vs Authorization Explained with Real-Life Examples
- **crypto.randomBytes for secure tokens** - [node.js official doc](#),
 - ▶ JWT Authentication using Refresh, Access Tokens - NodeJS

- **React Context API** - [passing data deeply w context](#),
 - ▶ Learn React Context Explained in 10 minutes - useContext ,
 - ▶ Introduction to React Context API | Complete React Course in Hindi #57
- **Axios interceptors / silent token refresh** - [axios interceptors doc](#),
 - ▶ Refreshing Tokens With Axios Interceptors
- **MongoDB TTL indexes** - [mongodb ttl indexes](#),
 - ▶ #58 Time to Live (TTL) Index | Understanding Index in MongoDB | MongoDB Complete Co..., [mongodb indexing](#),